

Stories from my Experiences Learning Scrum

Eric Sink

Software Craftsman

SourceGear

Champaign, Illinois, United States

eric@sourcegear.com

Abstract—An experienced (read: “old”) software developer recounts the ups and downs of learning Scrum as part of a geographically distributed team working on a pre-release product.

Keywords—component; software development; agile; scrum; continuous integration; burndown chart; collaboration; estimation; time tracking

I. A NEW PROJECT

In January of 2008, Jeff and I began working on a new project. Originally, we just called it an exploration, more like research than product development. We wanted it to eventually become SourceGear’s next big product. But we had a lot of work to do before we could even consider that.

Jeff lives in southern Indiana. I live in central Illinois. For quite a while, it was just the two of us.

What development methodology did we use? Well, we both have plenty of gray hair (or rather, I assume and hope that Jeff’s hair would be gray if he had any left). We’ve been working together since 1992. And even though it’s easy to look at our code and know which of us wrote it, we tend to have the same approach, in all the ways that matter.

So, our development methodology looked like this:

EVERY DAY, DO WHATEVER SEEMS
TO MAKE THE MOST SENSE.

And this mostly worked fine.

Eventually, our research project moved far enough along that our company decided to make it real. We decided to commit to make it a product and to name it Veracity¹. To do that, we needed to gradually migrate more of our developers onto the effort.

But Jeff and I are the wrong foundation for building a team. Neither of us is likely to ever win an award for our ability to work and play well with others. If our team of two became a team of ten, who would, you know, coordinate stuff? We would need somebody sort of manager-ish, right?

¹ Disclosure: Veracity has been developed entirely by my company, SourceGear, and I hope that we someday find a way to make money doing something related to it. If the mention of Veracity here offends you, remember that Veracity itself is open source (Apache License version 2.0). If you’re still bothered, keep in mind that there’s a very good chance I will fail to make any money and therefore end up selling my guitar collection to pay for ramen noodles. If you’re still not feeling better, please accept my apologies, and my suggestion that you dismiss me as the evil, money-grubbing entrepreneur that I am.

Jeff is, well, a nerd. To the core. He’s one of the best developers I have ever known, but he’s not really the manager type. To be fair, I guess we don’t really know if Jeff would be any good as a project manager. Similarly we don’t know if Gwyneth Paltrow would beat Mike Tyson at boxing. She’s never tried it. It never really occurred to anybody that she would.

But I’ve tried it. Er, management, not boxing. And it usually hasn’t gone very well. In my history with SourceGear, I have functioned well as a provider of vision or code, but my efforts to be a provider of structure or predictability have largely failed. In short, I’m a lousy manager. Most of my coworkers are polite enough not to say it that way, but I think we all know the score.

Regardless of what terminology you prefer, a team of ten people needs *somebody* who is thinking about those people.

- Who is doing what?
- What does everybody need?
- Are the obstacles getting cleared out of their way?

These are mostly questions about people, not code. It’s not really that I don’t like people. I just get so engrossed in the project that I forget about them.

So the first thing we had to fix about our team was to make it into a container where people could be placed safely. In March 2009, we added a third person, Ian. The idea was that Ian would initially join the team as a developer and become the project manager as the team grew. We wanted him there early so he could develop a lot of knowledge of the code before his time became consumed with helping others.

How did our methodology change with the addition of Ian? Not that much. The group was still very small. You can survive without much structure when there are only two or three people involved. We changed our methodology to:

EVERY DAY, DO WHATEVER SEEMS
TO MAKE THE MOST SENSE.
AND TRY TO REMEMBER TO
TALK TO EACH OTHER.

And this mostly worked fine.

II. AND THEN THERE WERE EIGHT.

In the fall of 2009, several other SourceGear developers got freed up all at the same time. We began transitioning them to join our team. Suddenly, our methodology seemed inadequate.

SourceGear had never really been big on any kind of formal process. We had some of the elements. We tend to do specs. We have reasonably formal QA. We have test plans, continuous integration, release cycles, and stuff like that. But nobody is really following any sort of documented process or methodology. So as the team was growing, we talked about maybe trying to follow some kind of method that is well-known. We asked ourselves if maybe we should try and use Scrum.

I recall the initial discussion about Scrum with only Ian, Jeff and myself. Ian was enthusiastic about Scrum. I was enthusiastic about not having to do Ian's job. And Jeff mumbled something about tracking down a stray pointer that was causing his code to dump core.

So we decided to give Scrum a try. In the course of writing this paper, we dug up Ian's slide deck from the team meeting where he announced our plan to use Scrum. It talks about pigs and chickens and sprints and backlogs and burndown charts and daily standups and the obligatory reference to Michigan football that he can't resist mentioning at every meeting.

Attitudes from the people on the team about Scrum seemed to fall in three groups:

1. People who thought Agile was a synonym for "No process" and "We don't have to do specs anymore".
2. People who thought Agile would be a strict, burdensome methodology, filled with TPS reports and all kinds of distractions to prevent us from building software.
3. Oh, and there was one guy who knew about Agile and liked the idea.

So, Scrum survived its first meeting at SourceGear and we kept moving forward. We tried to calm the people in

groups 1 and 2 by talking about the difference between discipline and ceremony. We talked about wanting to raise our level of discipline as a software team, but not to raise our level of ceremony. Nobody wants a team of ten to follow some sort of big-M methodology which involves more paperwork than software work.

We decided to proceed with Scrum and agreed to use the parts that work, ignore the parts that don't make sense, and try to get better improve our practices every sprint.

III. THE DAILY STANDUP

We began holding a 15-minute status meeting every day at 11am central time. At first, we had plenty of grumbling on the team about the idea of a meeting every day. Some of that grumbling might have come from me. I'm no fan of meetings. But our daily standup has turned out to be a useful thing without being too much of a disruption. We've mostly learned to exchange just the right amount of information. We use our daily standup to start discussions which then get taken offline.

We do try to follow a basic script, going around the room with each person giving a brief report:

- What I just did
- What I'm doing next
- What I need

We've had a wide diversity of styles for reports. Sometimes a report turns into a rambling stroll. And once in a while, a discussion breaks out and has to be killed mercilessly.

In my opinion, the most memorable report ever came from Brody, who had been working on the "merge" feature. When it came his turn to speak, he simply bellowed, "MERGE". That was it. One word. Actually, one syllable.

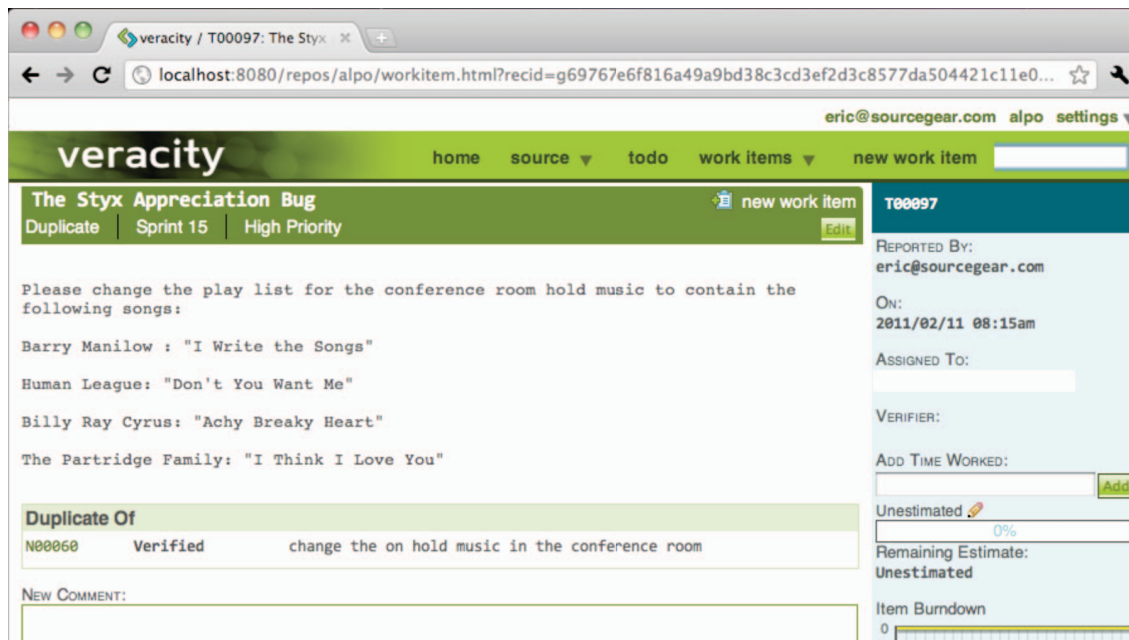


Figure 1

I wanted to tell Brody that if he had wanted to expend even less effort, he could probably have used the unvoiced form of the consonant. I think if we had all heard “MERSH”, we could have figured out what was going on and he wouldn’t have had to use quite so much energy.

Our daily standup is somewhat complicated by the fact that our team is not all in the same location. Most of us are in Champaign, Illinois, but we’ve got one developer in Indiana and another in Florida. So our Scrum meetings involve the use of a speakerphone. And this has triggered various colorful discussions about what kind of music is played in the virtual conference room while people are waiting for others to join the call.

For a very long time, the hold music was some sort of a classical piece. People complained, so I suggested we might switch to some classic Van Halen. People complained even more.

I don’t remember how we ended up switching to Styx, but that’s when the complaining soared to new heights. I thought the level of whining was way out of proportion, and tried to that there were plenty of worse things to listen to. Meanwhile, Troy, who runs our phone system and is also a developer on the team, wisely started refusing to change the music unless somebody logged a bug. So I added work item T00097 (see Figure 1).

To my amazement, people thought this was an improvement over the previous selections. But this playlist didn’t last long. Currently, those who dial in first for the daily standup are treated to a recording of Soviet spy transmissions from the cold war. None of us understand it because (a) we don’t speak Russian and (b) the messages are apparently encoded with a one-time pad, but everyone seems to agree that it’s better than Mr. Roboto.

Someday perhaps we’ll find some hold music that doesn’t actively discourage punctuality. In the meantime, I can still say that using the phone for the daily scrum has worked out better than I might have predicted. When Paul (in Florida) told us that he was leaving for a bit to ride his bike over to the beach so he could watch the launch of the space shuttle, it was nice for all of us on the call to be able to say, “I hate you” with just the right vocal inflection. Such subtleties can more easily get lost when using written forms of communication.

I have come to think of our daily standup as being similar to a security guard at a bank. Most security guards stand around for their entire career without ever firing their weapon. It’s probably a boring job. But the consistent presence of that security guard probably prevents some big problems from ever happening. Our daily standup is the same way. Nothing exciting ever really happens. But we can confidently assume that many big problems have been avoided because we regularly take the time to get synced up.

IV. WORKING TOGETHER

The culture of Scrum teams seems to be built on working together in shared spaces. In contrast, our company has always placed a high value on each person having a private office.

We are aware that there are tradeoffs here. A private office gives each person a quiet place to work, but it also creates the opportunity for people to get isolated. So even as we provide private offices, we create ways to drag people out of them, including soda in the kitchen, lunch together on Wednesdays, a pool table, and a video game room.

As we get involved with Scrum and begin hearing stories from other people, sometimes it seems like we are missing out on something by not using shared spaces. One of our first Scrum resources was an excellent video [1] made by Hamid Shojaei, founder of Axosoft². Last year Hamid moved his company into new office space which is configured specifically for Agile teams. Nobody has a private office, not even Hamid. Everyone sits in carefully designed work rooms, designed to accommodate six people.

At SourceGear, we haven’t been quite as brave. It feels like it would be so hard for us to make a radical change, for three reasons:

1. People are accustomed to having their own office. It would seem like taking something away from them.
2. It would cost us a pile of money to reconfigure our space. I’d be hesitant to spend that much cash on something that we would consider an experimental change.
3. Our offsite people wouldn’t be able to participate in the work rooms anyway.

So we have kept our offices, but we try to promote as much real-time interaction as possible.

One thing we did very early on was to try something we called “pair programming lite”. Ian grouped all the developers into pairs. We were not instructed to actually do real pair programming. Rather, we were simply to stay in greater contact, work together on features, review each other’s code, and so on. Not surprisingly, Ian assigned Jeff and I to be partners, probably because he didn’t think anybody else should have to put up with us.

This experiment didn’t work very well, as evidenced by (a) the fact that we stopped doing it, and (b) none of us seem to remember much about it. I had completely forgotten about “pair programming lite” until I was looking through old emails during the writing of this paper. I asked the other people on our team if they remembered anything. The only thing I got back was that Jeremy and Mary Jo were partners, and Jeremy used to bring her chocolate to appease her when he got too annoying.

It’s interesting how some agile practices are easily adapted for our circumstances and some are not.

Because we don’t work in shared spaces and we are geographically distributed, we do place a lot of emphasis on the between-iteration meetings. Everyone comes to Champaign, and we spend a lot of face time, reviewing what happened the previous sprint and talking about what should happen during the next one.

² Disclosure: I have no significant legal or financial connection with Axosoft. One of SourceGear’s products includes a feature that supports integration with one of Axosoft’s products. The two companies exchanged some marketing favors, but no money. Last time I saw Hamid we shared a cab to the airport.

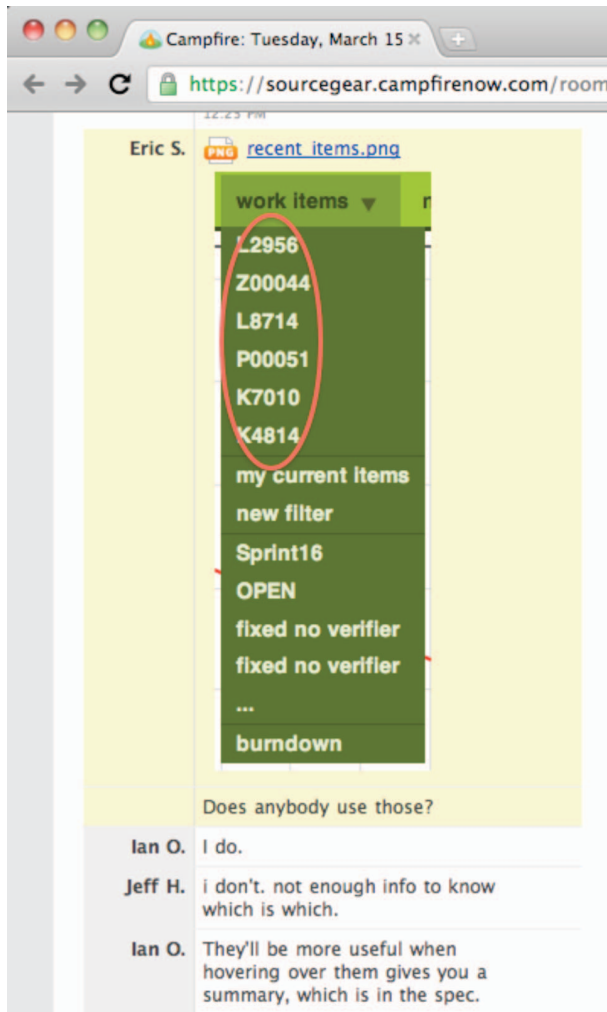


Figure 2

We also make use of an online chat room, Campfire [2], from 37signals³. We use the chat room to create a virtual environment that provides some of the benefits of working together in a shared physical space. Our basic guideline is that if you are working, you should be in the chat room.

Campfire has been an amazing resource for our team. We have little conversations there all throughout the day (see Figure 2). We ask technical questions. We talk about whose fault it was that the build broke. We make jokes and post lolcat pictures.

It may not be quite as cool as having a well-designed team work room, but it is working very well for our situation.

V. ITERATIONS

One of our first decisions was about how long our sprints should be. The Scrum gurus seem to always say that each iteration should be 2-3 weeks. I find their arguments to be

³ Disclosure: I have no financial or legal connection with 37signals. I just like their product.

compelling, but I also find it hard to imagine how that could work for our current situation. Our major stumbling block has been the notion of meaningful end-of-sprint releases.

- When the product has never been released to users, what is an end-of-sprint release?
- Exactly what are we releasing?
- To whom?
- How do we get a full development and QA cycle in the boundaries of a single sprint?

We currently have iterations of 4-5 weeks. But we have struggled with what it means for us to be “release-ready”. We didn’t even try doing end-of-sprint releases until after sprint 11. Then we began talking about being “release-ready” as more of a *relative* thing. During active development, the stability of the code fluctuates. We simply try to make sure that the end of the sprint is a time when things are *more* stable than they were in the middle of the sprint. As much as possible, tasks should be completed and stuff should be checked in to the repository. This is our way of implementing good practices without trying to force things to happen that don’t make sense.

Because Veracity is open source, even if we have struggled to figure out what “ready” means, the word “release” is pretty clear. At the end of every sprint, we package up the code and make it available for anyone who wants to see it. We label these code snapshots as “preview releases”, and they include source code only, no compiled executables. These previews give us the opportunity to practice release preparation, hoping that the picture will be more complete when the product is more mature.

VI. BURNDOWN CHARTS

For us, one of the most useful parts of Scrum is the burndown chart. By adopting the disciplines of planning each sprint, estimating each task, and tracking our time, we are rewarded with a pretty picture.

Let me try and explain that better.

The real issues here are estimation and time tracking. In general, programmers hate both of these. I am no exception. I don’t *want* to try and figure out how much time something is going to take. And I certainly don’t want to keep track of how I am spending my time. Just leave me and my compiler alone and when the task is done, we will tell you.

I believe my attitudes about such things are not atypical. As developers, we carefully hone our ability to give excuses for not doing things we don’t want to do.

I have come to think of estimation and time tracking as being similar to code coverage. People who have never really tried using code coverage tools are quick to point out all the reasons why code coverage is a waste of time. They can describe in great deal why code coverage will not solve all their problems. They know that even 100% code coverage does not guarantee that software is correct. All of their excuses are built on facts. But it is also difficult to find someone who has used code coverage who believes it is pointless.

Similarly, many of us can spout reasons why estimation and time tracking are a bad idea. It’s a hassle. It’s a waste of time. The estimates are never correct anyway. Steve



Figure 3

McConnell has an excellent book on software estimation [3]. But for some of us, the mere length of this book (308 pages) becomes part of our excuse not to do estimation. If it takes 308 pages to explain it, then it's gotta be more trouble than it's worth, right?

It is true that best practices can be taken too far. Many of them produce their greatest returns when applied in a small way. Nobody's going to be hiring me as an agile coach, but I've got more experience than I did two years ago, and I've learned some great lessons. Nowadays, when somebody tries to tell me that estimation and time tracking are pointless, I ask them, "Have you tried these practices with 4-week sprints and a burndown chart?"

This is obviously not my first time trying to use estimation and time tracking, but my previous experiences didn't work quite as well. Scrum is part of the reason why:

- *Short sprints* make estimation and time tracking easier.
- *Burndown charts* make estimation and time tracking worthwhile.

Estimation is really hard when you're trying to decide how much time it will take to do a task which is (a) eight months away, and (b) dependent on stuff that is supposed to be done in the seven months intervening. So don't do that. When something doesn't make sense, that might be a good indicator that you're doing it wrong. Don't try to claim that estimation is a waste of time until you've tried doing it in 4-week chunks.

And once you start doing estimates, then you get to use burndown charts, which, in my humble opinion, are deeply neat. The burndown chart gives us a visual representation of how our progress is going for any given sprint (see Figure 3).

The red line shows the ideal, what our work *should* look like. The green line shows reality. If we extrapolate the green line until it crosses the X axis, we can see when we could expect the tasks in the current sprint will be done, based on our progress so far.

Monitoring our burndown charts has allowed us to have a more realistic idea of when our product cycle will be complete. If we fail to finish the tasks for one sprint, something in the future is going to have to shift. Either the date moves back, or the feature list gets shorter.

Note that none of this stuff has been a panacea for us. There was a period of several months where every time we finished a sprint we had to insert a new one to deal with the stuff that didn't get done. Ian described sprint 15 as the first sprint where he didn't have to redo the 1.0 plan.

If I were being particularly pessimistic, I would observe that the only problem Scrum has solved is being unaware of our other problems. But that's still a pretty big deal. It is always better to confront reality than to avoid it.

VII. DOGFOODING

Throughout this development cycle, we have been building a tool to support Scrum at the same time as we are learning how to do Scrum.

When the team first got started, we used Jira [4] by Atlassian⁴. Later, we migrated all our project tracking to Veracity's Scrum module. Building tools for Scrum practitioners has forced us to think about Scrum in different ways than we would by simply practicing it.

⁴ Disclosure: I have no legal or financial connection with Atlassian. In fact, I suppose it would be accurate to say that they are a competitor, one for whom I have much admiration.

Sometimes this has been incredibly helpful. One of the things we did was build a dashboard that is integrated with our build system, so we can see at a glance how the continuous integration and nightly builds are getting along. We release a snapshot of the code every night if the nightly build passes all the automated tests (see Figure 4).

At other times, the duality of our situation has made things awkward. Sometimes we get in debates about how the more experienced Scrum teams would want our software to work. For example, we have a long-running argument about the “priority” field. Our company has been selling bug-tracking software for 10 years. Every tool we have ever written has had a priority field. All bug-tracking tools have a priority field. It is heresy to suggest not having one.

But if we’re doing Scrum, do we really need one? What would we use it for? When we plan sprints, we move stuff from the backlog into the current sprint. Once it’s in the sprint, it doesn’t really have a priority. It’s either in the sprint or it’s not.

I suppose the priority field *could* be more useful as we make decisions about what gets moved from the backlog into each sprint. But it’s not like we’re going to just sort by priority and take the top 10 items per person. The priority field contains one person’s subjective opinion of how important something was at some point in the past. When I’m triaging stuff for the next sprint, I want to know how important this work item is *now*.

Another recurring issue is about how much flexibility should be provided for people who get behind on their scrum planning and time tracking. And I’ll just go ahead and admit that I am one of the primary culprits.

During sprint 14, I basically did no planning or time tracking. I did plenty of coding, but in terms of making sure other people could see how my work was going, I did a terrible job. I promised to do better in sprint 15.

And then in sprint 15, I got a little behind. So I resolved to catch up. I started by creating work items for all of the

things that were supposed to be in my sprint 15 plan. And then I went to retroactively log time for those tasks starting at the first day of the sprint. But I found that Veracity wouldn’t let me log work for an item before the date it was created.

So once again our team ended up in a philosophical discussion of how Scrum should be done, blended with a discussion of how our software should work. It seemed to me that any Scrum team should want to provide amnesty for developers who actually want to get caught up.

Anyway, through a combination of increased discipline and cheating, I was able to get my sprint 15 burndown chart to be a fairly accurate reflection of what really happened. And I am pleased to report that I did a much better job planning and time tracking during sprint 16. There’s hope for me yet.

VIII. WE’RE NOT DONE YET

We’re going to continue trying to do Scrum better, improving with each iteration. Looking ahead, I see a number of places we can move forward.

A. End-of-sprint releases

I hope that after our first release to end users, we will be able to consider short sprints with a truly release-ready build at the end of each sprint.

B. Cooperating with the rest of the company

Currently, the pigs work mostly in isolation. We just haven’t had many chickens around. That will change as the product gets into the hands of end users. Our pigs and chickens will have to learn how to deal with each other.

C. Managing our backlog

We need to spend more time and attention between sprints when we prioritize our backlog. So far, it has been reasonably simple to figure out what is going to happen in

	Linux 32-bit Debug	Linux 32-bit Release	Linux 64-bit Debug	Linux 64-bit Release	Mac 32-bit Debug	Mac 32-bit Release	Mac 64-bit Debug	Mac 64-bit Release	Windows 32-bit Debug	Windows 32-bit Release	Windows 64-bit Debug	Windows 64-bit Release
0.7.0.2752	Queued On: Mar 22 at 16:12:04	Building Started: Mar 22 at 16:14:12 ETA: 55m 28s	Queued On: Mar 22 at 16:12:00	Queued On: Mar 22 at 16:11:31	Building Started: Mar 22 at 16:20:22 ETA: 36m 3s	Build Failed On: Mar 22 at 16:20:14	Build Failed On: Mar 22 at 16:21:01	Build Failed On: Mar 22 at 16:17:41	Queued On: Mar 22 at 16:12:12	Testing Started: Mar 22 at 16:14:47 ETA: 27m 18s	Queued On: Mar 22 at 16:12:08	Queued On: Mar 22 at 16:11:39
0.7.0.2751	Skipped On: Mar 22 at 16:14:16	Success Finished: Mar 22 at 16:13:32 Duration: 1h 3m 22s	Testing Started: Mar 22 at 15:33:35 ETA: 11m 12s	Success Finished: Mar 22 at 15:32:59 Duration: 55m 38s	Success Finished: Mar 22 at 16:00:52 Duration: 40m 5s	Success Finished: Mar 22 at 15:20:10 Duration: 36m 3s	Success Finished: Mar 22 at 15:37:29 Duration: 28m 36s	Success Finished: Mar 22 at 15:08:16 Duration: 21m 47s	Success Finished: Mar 22 at 16:14:20 Duration: 15m 20s	Success Finished: Mar 22 at 15:26:25 Duration: 35m 14s	Testing Started: Mar 22 at 15:36:05 ETA: 5m 9s	Success Finished: Mar 22 at 15:35:29 Duration: 39m 4s
0.7.0.2750	Skipped On: Mar 22 at 15:10:32	Skipped On: Mar 22 at 15:10:24	Skipped On: Mar 22 at 14:37:44	Skipped On: Mar 22 at 14:37:52	Skipped On: Mar 22 at 14:44:25	Skipped On: Mar 22 at 14:44:22	Skipped On: Mar 22 at 14:46:41	Skipped On: Mar 22 at 14:46:37	Skipped On: Mar 22 at 14:51:23	Skipped On: Mar 22 at 14:51:15	Skipped On: Mar 22 at 14:59:38	Skipped On: Mar 22 at 14:59:33
0.7.0.2749	Skipped On: Mar 22 at 15:10:50	Skipped On: Mar 22 at 15:10:42	Skipped On: Mar 22 at 14:37:56	Skipped On: Mar 22 at 14:37:48	Skipped On: Mar 22 at 14:44:14	Skipped On: Mar 22 at 14:44:18	Skipped On: Mar 22 at 14:46:33	Success Finished: Mar 22 at 14:45:43 Duration: 22m 30s	Skipped On: Mar 22 at 14:51:30	Skipped On: Mar 22 at 14:51:27	Skipped On: Mar 22 at 14:59:29	Success Finished: Mar 22 at 14:58:49 Duration: 35m 45s
0.7.0.2748	Skipped On: Mar 22 at 15:10:50	Success Finished: Mar 22 at 15:10:42	Skipped On: Mar 22 at 14:37:56	Skipped On: Mar 22 at 14:37:48	Skipped On: Mar 22 at 14:44:14	Success Finished: Mar 22 at 14:44:18	Skipped On: Mar 22 at 14:46:33	Skipped On: Mar 22 at 14:46:37	Skipped On: Mar 22 at 14:51:23	Success Finished: Mar 22 at 14:51:15	Skipped On: Mar 22 at 14:59:38	Skipped On: Mar 22 at 14:59:33

Figure 4

each sprint. We are dogfooding. Each sprint, we fix the stuff that doesn't work. In the future, more voices and stakeholders will be involved. The process of planning each sprint will change and will [I think] become a bit more typical of Scrum practices.

D. Making the Product Owner role more meaningful

My role will evolve into something more like the Product Owner. This Scrum role has not really existed for us, at least not in the way I think it usually does. In terms of vision and direction, Veracity has a lot of me in it, but I've been working primarily as a coder. In the future, I will be stepping off the development team and working more in the non-programming roles.

E. Interacting with other agile practitioners

As we continue gaining experience with agile development, we are interested in the degree to which those who use agile practices function as a community.

In the past, our company has a great deal of experience working within the Microsoft .NET community. We found partners and friends there, people with whom we could consult or commiserate. Looking to the future, we have found ourselves wondering: Where do developers find community without being religiously devoted to a platform?

REFERENCES

- [1] <http://www.youtube.com/watch?v=Q5k7a9YEoUI>
- [2] <http://campfirenow.com/>
- [3] McConnell, Steve. "Software Estimation: Demystifying the Black Art", Microsoft Press, 2006.
- [4] <http://www.atlassian.com/software/jira/>